

3-4 · DuckDB를 이용한 데이터 분석 · 실습 1.5H

# SQL 집계·조인 분석 · 윈도우 함수와 CTE

파일끼리 JOIN 하고 Pandas 답과 대조

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

# 1.5시간 뒤에는

---

- ① Parquet 와 CSV 를 적재 없이 곧바로 JOIN 한다
- ② SQL 결과와 Pandas 결과의 diff 가 0 인지 교차 검증한다
- ③ CTE + 윈도우 함수로 누적 매출·이동 평균을 낸다
- ④ PARTITION BY + QUALIFY 로 지역별 상위 3명을 뽑는다

일곱 단계 모두 `duckdb.sql(""" ... """)` 한 호출씩 — 확인 포인트를 지나며 진행합니다

# 일곱 단계 — 파일 세 개가 있는 폴더에서

3-3 의 events.parquet, 3-1 의 orders\_step1.parquet, 3-2 의 customers.csv 가 작업 폴더에 있어야 합니다

| 1. 상위 지출            | 2. 파일 JOIN              | 3. 교차 검증             | 4·5. CTE·윈도       | 6·7. 프레임·편의               |
|---------------------|-------------------------|----------------------|-------------------|---------------------------|
| GROUP BY<br>LIMIT 5 | parquet + csv<br>지역별 매출 | Pandas 와<br>diff = 0 | 누적 매출<br>지역별 상위 3 | 이동 평균 ma7<br>GROUP BY ALL |

# 핵심 질의 — 상위 지출 사용자와 파일 JOIN

|         |                                                            |
|---------|------------------------------------------------------------|
| 행 거르기   | WHERE event_type = 'buy'                                   |
| 묶기      | GROUP BY user_id                                           |
| 정렬      | ORDER BY spend DESC LIMIT 5                                |
| 파일 JOIN | FROM 'orders_step1.parquet' AS o JOIN 'customers.csv' AS c |
| 잇는 키    | ON o.customer_id = c.customer_id                           |
| 지역별 매출  | sum(o.price * o.quantity) AS total ... GROUP BY c.region   |

확인: 1단계는 5행이 spend 내림차순 · 2단계는 지역 4행 — price 결측 30건은 SQL 의 sum 도 건너뛴다

## 3단계 — Pandas 답과 diff 를 뺀다

1

### Pandas 로 같은 질문

orders\_enriched.parquet 를 groupby('region') 으로 orders.total

2

### region 으로 merge

suffixes=('\_sql', '\_pd') 로 붙이고 diff = total\_sql - total\_pd

3

### diff 가 전부 0 이면

두 도구의 답이 같다 — 0 이 아니면 결측을 다루는 시점을 의심

이 방법의 이름

교차 검증

3-5 에서 Pandas 를 DuckDB 로 옮길 때 결과가 안 바뀌었는지 확인하는 유일한 방법

# CTE 로 두 문단 — 누적 매출과 지역별 상위 3

## 4단계 — 누적 매출

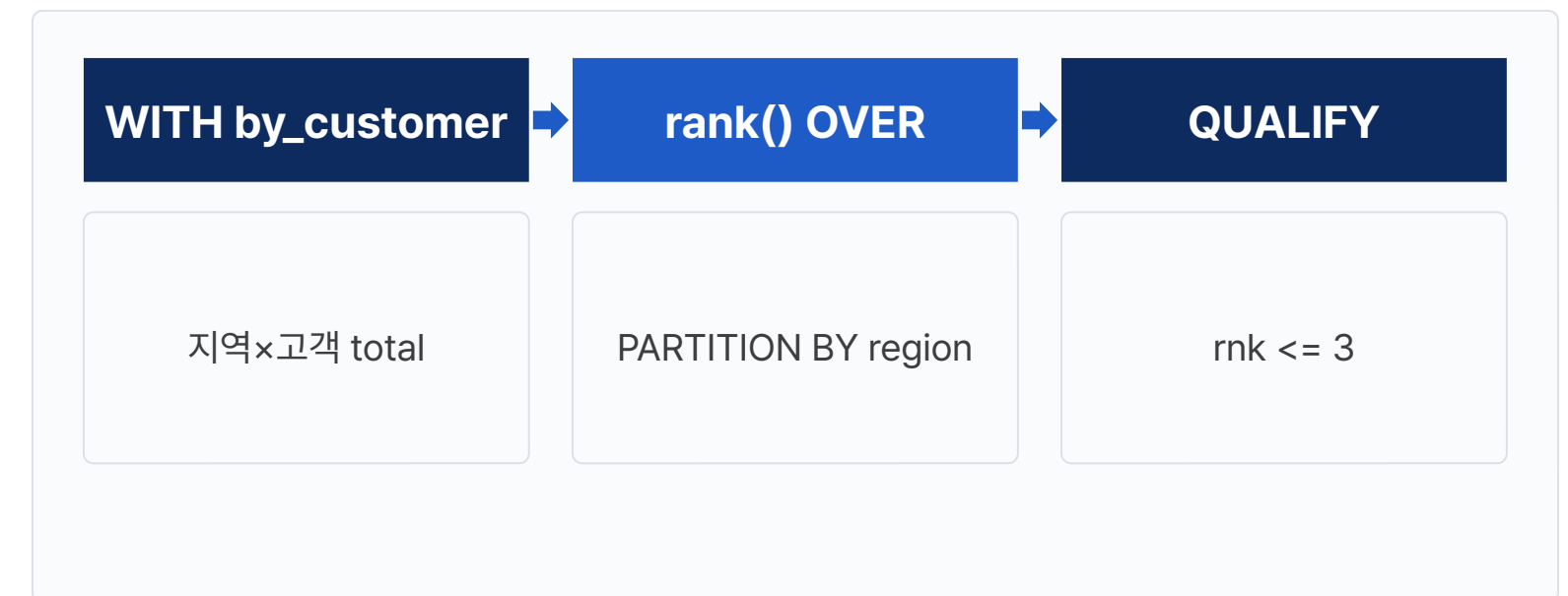
daily 를 만들고 sum(revenue) OVER (ORDER BY d)



VS

## 5단계 — 지역별 상위 3

by\_customer 위에 rank() 와 QUALIFY rnk <= 3



# WHERE 에 윈도 별칭을 써 본다 — Binder Error

## 실패 — WHERE `rnk <= 3`

별칭 `rnk` 를 찾지 못한다는 Binder Error

- SQL 은 WHERE 를 SELECT 보다 먼저 평가한다
- SELECT 에서 지은 `rnk` 를 WHERE 가 모른다
- 2-2 에서 CTE 로 감쌌던 바로 그 이유

VS

## 해결 — QUALIFY 또는 CTE

윈도 결과는 QUALIFY, 어디서나는 CTE

- QUALIFY `rnk <= 3` — DuckDB 한 줄
- ranked AS (...) 로 감싸고 WHERE `rnk <= 3`
- 앞 결과와 같다 — 동물이면 3명을 넘을 수 있다

# 같은 이동 평균인데 — 앞쪽 6일이 다르다

## Pandas rolling(7) — 3-2

창이 안 차면 NaN — 앞 6일이 비어 있다

- 창 7일이 다 차야 값이 나온다
- 앞 6일은 NaN
- 3-2 에서 본 그 결과

VS

## SQL 윈도우 프레임

있는 만큼 평균 — 앞쪽도 값이 나온다

- ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
- 앞쪽 며칠은 그날까지의 평균
- 어느 쪽이 원하는 결과인지 정해야 한다

# CTE 를 떼어 보는 습관 · DuckDB 편의 문법

## 1 daily 만 따로 실행

date\_trunc('day', ts) 로 묶은 SELECT 만 LIMIT 3 — 3행이 나오면 통과

## 2 어느 단계에서 틀어졌는지

긴 질의가 이상하면 CTE 를 하나씩 떼어 확인한다 — 바로 보인다

## 3 GROUP BY ALL

집계가 아닌 열로 알아서 묶는다 — 다른 DB 로 옮길 때는 풀어 쓴다

## 4 EXCLUDE · COLUMNS

SELECT \* EXCLUDE (event\_id) 몇 개만 빼고 전부 · max(COLUMNS('^value\$')) 패턴 집계

「같은 분석인데 숫자가 다르다」의 원인은 6단계 같은 미묘한 차이 — 파이프라인을 옮길 때 반드시 확인

# 이 1.5시간이 끝날 때 확인할 것

---

- 1 1단계 5행이 spend 내림차순, 2단계 지역 4행
- 2 merged 의 diff 가 전부 0 이다
- 3 cum\_revenue 가 매 행 revenue 를 더해 단조 증가한다
- 4 지역마다 rnk 1~3 고객 — QUALIFY 와 CTE 결과가 같다
- 5 ma7 앞쪽 며칠이 Pandas 와 다른 이유를 말할 수 있다

3-4 · 실습 1.5H 마무리

## 다음은 Pandas 연동 분석 (1H)

SQL 로 줄이고 DataFrame 으로 받아 등급을 매기고, 다시 SQL 로 질의하는 왕복을 이어 갑니다

✓ LIMIT 100 결과를 .df() 로 받아 pd.cut

✓ FROM top — DataFrame 을 SQL 로

✓ SQL → Pandas → SQL 세그먼트 분석

✓ 줄이기 전 .df() 의 시간·메모리 실측

가져갈 것

events.parquet 와 열린 파이썬 세션